4. Semantic Analysis

Syntax Directed Definitions – Syntax Directed Translation Schemes – Type Checking – Intermediate Code Generation

Andrea Polini, Luca Tesei

Linguaggi e Compilatori
Corso di Laurea in Informatica
Università di Camerino

ToC

1. Semantic Analysis: the problem
2. Syntax Directed Definitions
3. Syntax Directed Translation Schemes
4. Preliminaries
5. Types
6. Control Flow

Where we are?

So far we were able to check:

- the program includes correct “words”
- “words” are combined in correct “sentences”

What’s next?

- We would like to perform additional checks to increase guarantees of correctness
- We would like to transform the program from the source language into the target one, and according to precisely defined semantic rules

Where we are?

So far we were able to check:

- the program includes correct “words”
- “words” are combined in correct “sentences”

What’s next?

- We would like to perform additional checks to increase guarantees of correctness
- We would like to transform the program from the source language into the target one, and according to precisely defined semantic rules

Where we are?

So far we were able to check:

- the program includes correct “words”
- “words” are combined in correct “sentences”

What’s next?

- We would like to perform additional checks to increase guarantees of correctness
- We would like to transform the program from the source language into the target one, and according to precisely defined semantic rules

Additional checks

Additional Checks

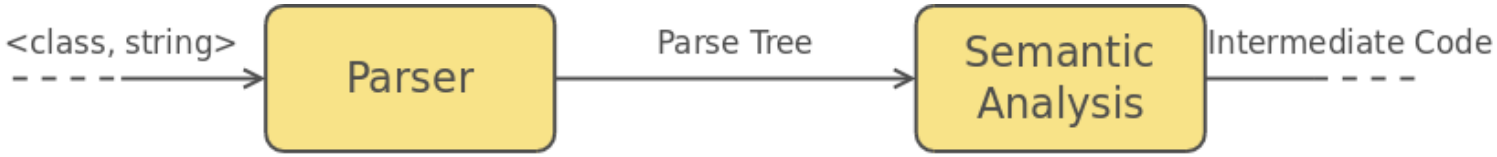
There are many additional checks that can be performed to increase correctness of code:

- Coherent usage of variables
  - definition-usage
  - type
- Existence of unreacheable code blocks
- ...

Semantic Analysis

In semantic analysis context sensitive analysis are performed without resurrecting to Context Sensitive grammar definitions. Here we focus on mechanisms for type checking and generation of intermediate code

Semantic analysis



ToC

1. Semantic Analysis: the problem
2. Syntax Directed Definitions
3. Syntax Directed Translation Schemes
4. Preliminaries
5. Types
6. Control Flow

Syntax Directed Definitions

Attributes

Attributes are used to associate characteristics and store values associated to grammar symbols.
A syntax directed definition provides the semantic rules to permit the definition of the values for the attributes

PRODUCTION	SEMANTIC RULE
$E \rightarrow E_1 + T$	$E.code = E_1.code \| \| T.code \| \| '+'$

- attributes are associated to grammar symbols and can be of any kind
- rules are associated to productions

Attributes

An SDD can be defined using two different kinds of attributes:

- Synthesized attributes: a synthesized attributes at node N is defined only in terms of attribute values at the children of N and at N itself
- Inherited attributes: an inherited attribute at node N is defined only in terms of attribute values at N's parent, N itself, and N's siblings

Attributes

Example

Consider the usual grammar and let's define a set of "reasonable" semantic rules:

$L \rightarrow E \quad E \rightarrow E + T \quad E \rightarrow T \quad T \rightarrow T * F \quad T \rightarrow F \quad F \rightarrow (E) \quad F \rightarrow id$

SDD and parse trees

An SDD with only synthesized attributes is called *S-attributed*

It is generally useful to represent attributes within parse trees. A parse tree showing the values of attributes is referred as an annotated parse tree

Order of evaluation for attributes

The order of evaluation of attributes should reflect the defined parsing strategy. In any case semantic rules impose an order of evalution that in case, inherited and synthetized attributes are present at the same time, is not guaranteed to exist.

Let's consider the expression "(3+4)*(5+6)" and let's derive its annotated parse tree from the semantic rules defined before

Inherited attributes example

Let's consider the non left recursive and factored grammar for expressions:
$E \rightarrow TE' \;\; E' \rightarrow +TE'|\epsilon \;\; T \rightarrow FT' \;\; T' \rightarrow *FT'|\epsilon \;\; F \rightarrow (E)|id$
define an SDD using as reference the parse tree for the sentence "$3 + 5 * 6$"

Evaluation Orders for SDD's

Dependency Graphs

A dependency graph represents the flow of information among the attribute instances in a particular parse tree.

- each attribute for a grammar symbol constitute a node in the graph
- syntesized attributes
- inherited attributes

Let's identify the dependency graph for the parse tree defined before, and let's compute the value of the various attributes

SDD with acyclic topological sort

S-attributed

If every attribute is synthesized the SDD is said S-attributed, in such a case an LR parser could even avoid the explicit derivation of the parse tree

L-attributed

Each attribute in the SDD satisfies one of the following conditions:

- it is synthesized
- it is inherited but it depends only from attributes on siblings on the left or inherited attributes associated to the parent symbol
- it is inherited or synthesized from attributes from the same symbol in a way that cycles are not generated

Semantic rules with controlled side effects

Side effects

A side effect consists of program fragment contained with semantic rules. It is necessary to control side effects is SDD in two possible ways:

- Permit incidental side effects
- Constraint admissible evaluation orders so to have the same translation with any admissible order.

Why to use them?

- to associate actions to carry on with specific steps of the compiler
- to print messages for the user useful during compilation
- to check correctness related aspects (e.g. types)

Semantic rules with controlled side effects

Side effects

A side effect consists of program fragment contained with semantic rules. It is necessary to control side effects is SDD in two possible ways:

- Permit incidental side effects
- Constraint admissible evaluation orders so to have the same translation with any admissible order.

Why to use them?

- to associate actions to carry on with specific steps of the compiler
- to print messages for the user useful during compilation
- to check correctness related aspects (e.g. types)

Semantic Rules with side effects

Example

Let's consider the following grammar:

$D \rightarrow TL; \quad T \rightarrow \textbf{int}|\textbf{float} \quad L \rightarrow L_1, \textbf{id}|\textbf{id}$

Let's add sematic rules to successively permit type checking

Semantic Rules with side effects

Exercise

Let's consider the following grammar that generates binary numbers with a decimal point:

$S \rightarrow L.L|L \quad L \rightarrow LB|B \quad B \rightarrow 0|1$

Design an L-attributed and an S-attributed SDD to make the translation in decimal numbers

Abstract Syntax Tree

Abstract Syntax Tree

Abstract Syntax Tree (AST), or just syntax tree, is a tree representation of the abstract syntactic structure of source code written in a programming language. Each node of the tree denotes a construct occurring in the source code. The syntax is "abstract" in not representing every detail appearing in the real syntax. For instance, grouping parentheses are implicit in the tree structure, and a syntactic construct like an if-condition-then expression may be denoted by means of a single node with three branches.
Syntax trees are useful for translation purpose making the phase much easier.

Let's consider the sentence $(a + b) * 5$ over the grammar:

$E \rightarrow TE' \quad E' \rightarrow +TE'|\epsilon \quad T \rightarrow FT' \quad T' \rightarrow *FT'|\epsilon \quad F \rightarrow (E)|\mathbf{id}|\mathbf{num}$

Let's build the parse tree and the AST

Using SDDs to build AST

To build a syntax tree two different kind of nodes need to be created, the leaves ($Leaf(op, val)$) and the internal nodes ($Node(op, c_1, \ldots, c_n)$). In the following consider the sentence $a - 4 + c$.

1. Let's built an SDD with actions permitting to derive the syntax tree for expressions grammar in the form suitable for LR parsing.
   $E \rightarrow E_1 + T$, $E \rightarrow E_1 - T$, $E \rightarrow T$, $T \rightarrow (E)$, $T \rightarrow$ **id**, $T \rightarrow$ **num**
2. Let's repeat the exercise for an expression grammar parsable by LL parsers.
   $E \rightarrow TE'$, $E' \rightarrow +TE'_1$, $E' \rightarrow -TE'_1$, $E' \rightarrow \epsilon$, $T \rightarrow (E)$, $T \rightarrow$ **id**, $T \rightarrow$ **num**

Towards type checking

Let's now consider the case of a grammar for type definition:
$T \rightarrow BC$, $B \rightarrow$ **int**, $B \rightarrow$ **float**, $C \rightarrow$ [**num**]C, $C \rightarrow \epsilon$
Define sematics rules to assign a type to an expression and try it on the sentence:

int[2][3]

ToC

1. Semantic Analysis: the problem
2. Syntax Directed Definitions
3. Syntax Directed Translation Schemes
4. Preliminaries
5. Types
6. Control Flow

Syntax Directed Translation

Syntax Directed Translation

A Syntax Directed Translation scheme permits to embed program fragments, called semantic actions, within production bodies. An SDT is a context-free grammar with program fragments embedded within production bodies.

- SDTs are an alternative approach to SDDs
- an STD is like an SDD except that the order of evaluation of the semantic rules is explicitly specified
- program fragments embedded within productions in curly braces are called semantic actions:

$$rest \rightarrow + \; term \; \{\text{print}('+')\} \; rest_1$$

Syntax Directed Translation

Construction

Any SDT can be implemented by first building a parse tree and then performing the actions in a left-to-right depth-first order.

However, SDT are typically implemented during parsing without the need to build a parse tree:

- introduce distinct marker nonterminals M_i in place of each embedded action;
- each marker has only one production $M_i \rightarrow \epsilon$.
- If the grammar with marker nonterminals can be parsed by a given method, then the SDT can be implemented during parsing

Syntax Directed Translation

STDs can be easily used to implement two important classes of SDDs:

- grammar LR-parsable and SDD S-attributed
- grammar LL-parsable and SDD L-attributed

In both cases the semantic rules of the SDD can be converted into an STD with actions that are executed at the right time.

During parsing an action in a production body is executed as soon as all the grammar symbols to the left of the action have been matched.

Postfix translation schemes

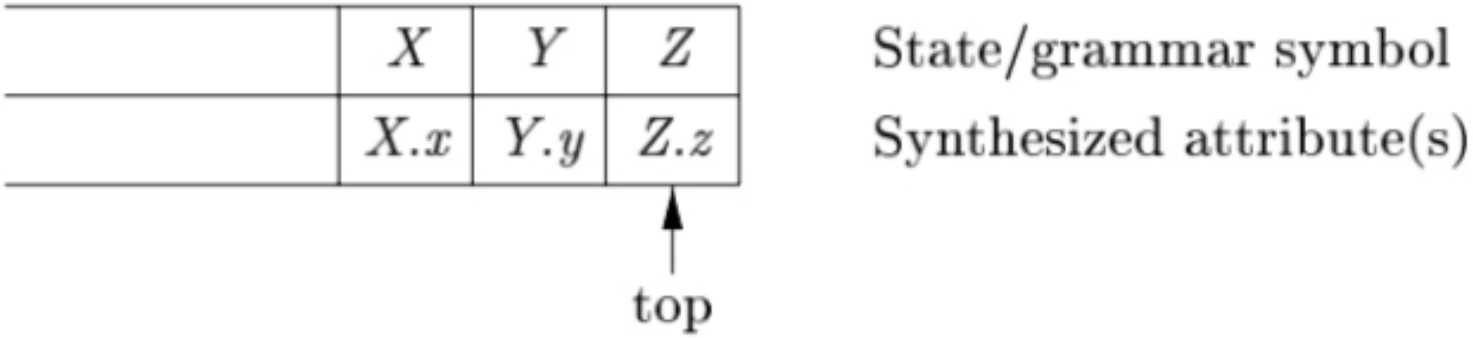
Simplest situation: bottom-up parsing with S-attributed SSD. In that case all the actions in the SDT are placed at the end of the production bodies. (Postfix SDT)

implementation

Postfix SDT are easy to implement with additional attributes for the stack cell. In particular it is useful to associate to each non-terminal on the stack the values assumed by the corresponding attributes.

Postfix translation schemes

For instance if you have a production like $A \rightarrow XYZ$ with a postfix SDT you will apply the actions in the SDT just before reducing XYZ to A. Stack elements will be complex or include pointers to complex data structures.



SDT with actions inside productions

Consider the production $B \rightarrow X\{a\}Y$. When do we perform the action inside the production?

- if the parse is bottom-up then we perform the action 'a' as soon as this occurrence of X appears on top of the parsing stack
- if the parse is top-down we perform 'a' just before we attempt to expand the occurrence of Y (non terminal) or check for Y on input (terminal)

Imagine each SDT fragment as a distinct non-terminal M with the only production $M \rightarrow \epsilon$

Implementing SDT

Not all SDT can be implemented during parsing

General implementation rules

Any SDT can be implemented as follows:

- Ignore the actions and parse the input to produce a parse tree
- Examine each interior node, say a production $A \rightarrow \alpha$. Add additional children to N for the actions in α, so the children of N from left to right have exactly the symbols and actions of α
- Perform a preorder traversal of the tree, and as soon as a node labeled by actions is visited, perform that action

Implementing SDT

Not all SDT can be implemented during parsing

General implementation rules

Any SDT can be implemented as follows:

- Ignore the actions and parse the input to produce a parse tree
- Examine each interior node, say a production $A \rightarrow \alpha$. Add additional children to N for the actions in α, so the children of N from left to right have exactly the symbols and actions of α
- Perform a preorder traversal of the tree, and as soon as a node labeled by actions is visited, perform that action

SDT and Top-Down parsing

Note: Including semantic actions in grammars conceived for being parsable by top-down strategies can be complicated

Question: Would it be possible to define semantic actions and then transform the grammar?

Eliminating Left Recursion (simple case)

- In case included actions just need to be performed in the same order then it is enough to treat them as terminal symbols
  $E \rightarrow E + T\{print('+');\}$
  $E \rightarrow T$

When an SDT computes attributes we need to be more careful.

Eliminating Left Recursion (general case)

It is always possible to transform a recursive grammar with actions if it is S-attributed.

In particular given the grammar with actions:

$$A \to A_1 Y \quad \{A.a = g(A_1.a, Y.y)\}$$
$$A \to X \quad \{A.a = f(X.x)\}$$

Consider the parse tree fragment for a derivation:

$$\ldots A \ldots \xrightarrow{*} \ldots XYY \ldots$$

It is possible to rewrite it in an equivalent one according to the following schema:

$$A \to X \quad \{R.i = f(X.x)\} \quad R \quad \{A.a = R.s\}$$
$$R \to Y \quad \{R_1.i = g(R.i, Y.y)\} \quad R_1 \; \{R.s = R_1.s\}$$
$$R \to \epsilon \quad \{R.s = R.i\}$$

Eliminating Left Recursion (general case)

It is always possible to transform a recursive grammar with actions if it is S-attributed.

In particular given the grammar with actions:

$$A \rightarrow A_1 Y \quad \{A.a = g(A_1.a, Y.y)\}$$
$$A \rightarrow X \quad \{A.a = f(X.x)\}$$

Consider the parse tree fragment for a derivation:

$$\ldots A \ldots \stackrel{*}{\rightarrow} \ldots XYY \ldots$$

It is possible to rewrite it in an equivalent one according to the following schema:

$$A \rightarrow X \quad \{R.i = f(X.x)\} \quad R \quad \{A.a = R.s\}$$
$$R \rightarrow Y \quad \{R_1.i = g(R.i, Y.y)\} \quad R_1 \ \{R.s = R_1.s\}$$
$$R \rightarrow \epsilon \quad \{R.s = R.i\}$$

SDT for L-attributed definitions

Assuming a pre-order traversal of the parse tree we can transform a L-attributed SDD in a SDT as follows:

1. action computing inherited attributes must be computed before the occurrence of the non terminal. In case of more inherited attributes for the same non terminal order them as they are needed
2. actions for computing synthesized attributes go at the end of the production

Example

Consider the production:

$$S \rightarrow \textbf{while}\ (C)\ S_1$$

assuming the "traditional" semantics for this statement let's generate the intermediate code assuming a three-address code where three control flow statements are generally used:

- `ifFalse` x `goto L`
- `ifTrue` x `goto L`
- `goto L`

Example

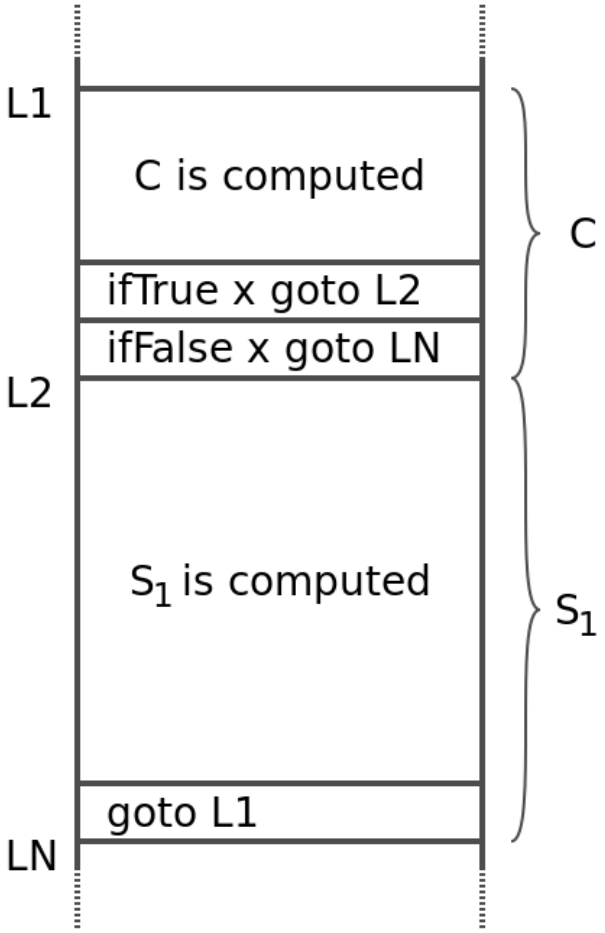
Consider the production:

$$S \rightarrow \textbf{while}\ (C)\ S_1$$

assuming the "traditional" semantics for this statement let's generate the intermediate code assuming a three-address code where three control flow statements are generally used:

- `ifFalse` x `goto L`
- `ifTrue` x `goto L`
- `goto L`

Intermediate Code Structure



while statement - rationale

The following attributes can be used to derive the translation:

- *S.next*: labels the beginning of the code to be executed after *S* is finished
- *S.code*: sequence of intermediate code steps that implements the statement *S* and ends with *S.next*
- *C.true*: label for the code to be executed if *C* is evaluated to true
- *C.false*: label for the code to be executed if *C* is evaluated to false
- *C.code*: sequence of intermediate code steps that implements the condition *C* and jumps to *C.true* or to *C.false* depending on the evaluation

while statement - SDD and SDT

SDD

$$
\begin{array}{ll}
S \rightarrow \textbf{while}\ (C)\ S_1 & L1 = new();\\
& L2 = new();\\
& S_1.next = L1;\\
& C.false = S.next;\\
& C.true = L2\\
& S.code = \textbf{label}||L1||C.code||\textbf{label}||L2||S_1.code
\end{array}
$$

while statement - SDD and SDT

Note for the translation:

- L_1 and L_2 can be treated as synthesized attributes for dummy nonterminals and can be assigned to the first action in the production

SDT

$S \rightarrow$ **while** (	$\{L1 = new(); L2 = new(); C.false = S.next;$
	$C.true = L2; \}$
C)	$\{S_1.next = L1; \}$
S_1	$\{S.code =$ **label**$\|\|L1\|\|C.code\|\|$**label**$\|\|L2\|\|S_1.code\}$

while statement - SDD and SDT

Note for the translation:

- L_1 and L_2 can be treated as synthesized attributes for dummy nonterminals and can be assigned to the first action in the production

SDT

$S \rightarrow$ **while** (	$\{L1 = new(); L2 = new(); C.false = S.next;$
	$C.true = L2;\}$
C)	$\{S_1.next = L1;\}$
S_1	$\{S.code =$ **label**$\|\|L1\|\|C.code\|\|$**label**$\|\|L2\|\|S_1.code\}$

Implementing L-attributed SDD

Translation can be performed according to two different strategies:

- traversing a parse tree
- during parsing

Traversing a parse tree

- Build the parse tree and annotate; if the SDD is not circular there is at least an order of execution that works
- Build the parse tree, add actions, and execute the actions in preorder; e.g. L-attributed SDDs translated into SDTs

During parsing

- Use a recursive descent parser
- Generate code on the fly
- Implement an SDT in conjunction with an LL-parser
- Implement an SDT in conjunction with an LR-parser

Implementing L-attributed SDD

Translation can be performed according to two different strategies:

- traversing a parse tree
- during parsing

Traversing a parse tree

- Build the parse tree and annotate; if the SDD is not circular there is at least an order of execution that works
- Build the parse tree, add actions, and execute the actions in preorder; e.g. L-attributed SDDs translated into SDTs

During parsing

- Use a recursive descent parser
- Generate code on the fly
- Implement an SDT in conjunction with an LL-parser
- Implement an SDT in conjunction with an LR-parser

Implementing L-attributed SDD

Translation can be performed according to two different strategies:

- traversing a parse tree
- during parsing

Traversing a parse tree

- Build the parse tree and annotate; if the SDD is not circular there is at least an order of execution that works
- Build the parse tree, add actions, and execute the actions in preorder; e.g. L-attributed SDDs translated into SDTs

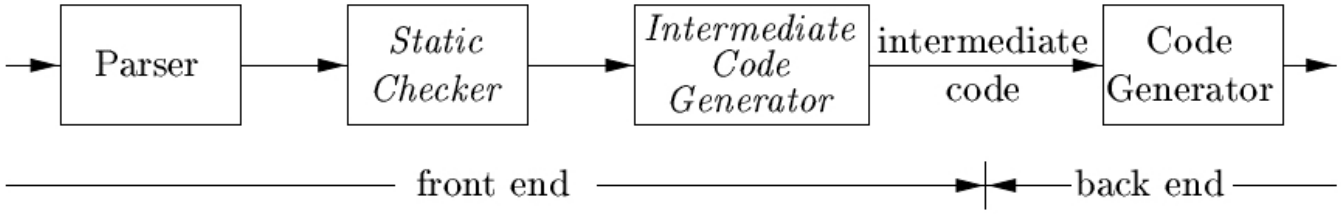
During parsing

- Use a recursive descent parser
- Generate code on the fly
- Implement an SDT in conjunction with an LL-parser
- Implement an SDT in conjunction with an LR-parser

ToC

1. Semantic Analysis: the problem
2. Syntax Directed Definitions
3. Syntax Directed Translation Schemes
4. Preliminaries
5. Types
6. Control Flow

Intermediate Code generation



- Last block in the front end of a compiler. We will consider:
  - intermediate representations – memory management is still abstracted
  - static checking – type checking in particular
  - intermediate code generation – the C programming language is often selected as an intermediate form because it is flexible, it compiles into efficient machine code and its compilers are widely available.

Intermediate Representations

The two most important intermediate representations are:

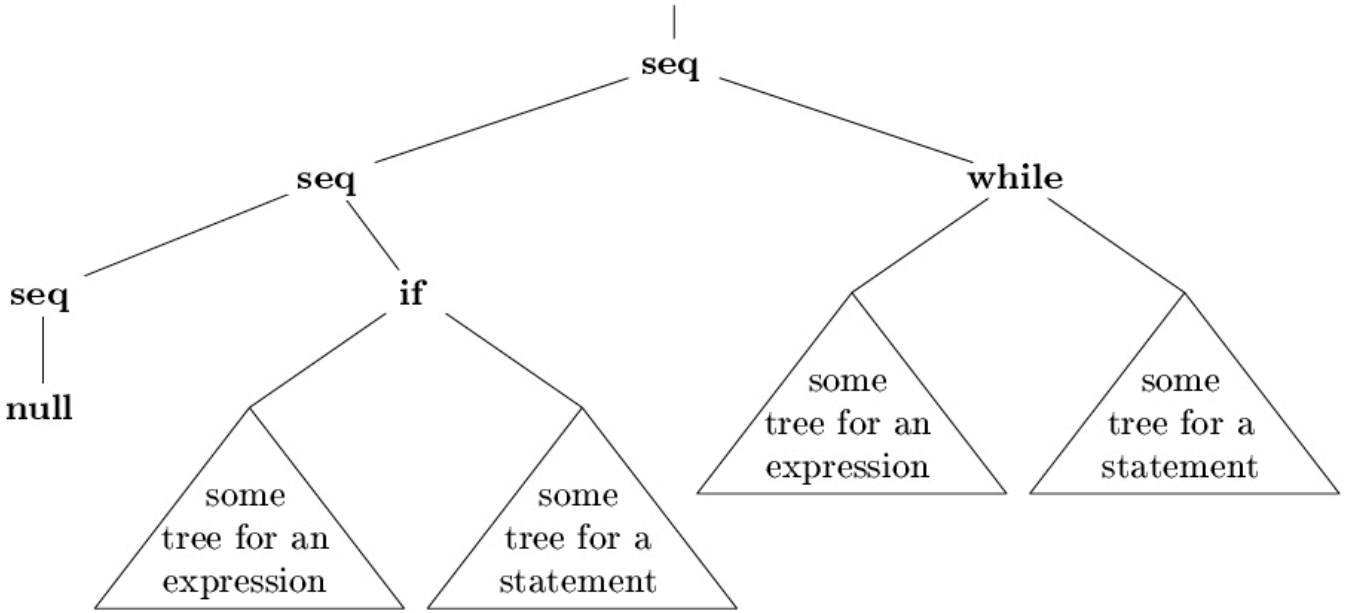
- Trees: parse trees, (abstract) syntax trees
- Linear representations: three-address code

Tree Representations

- (Abstract) syntax trees can be generated during parsing using a synthesized attribute
- Let's see an example for a basic *while language*

Tree Representations

- Example of syntax tree



Concrete vs Abstract Syntax

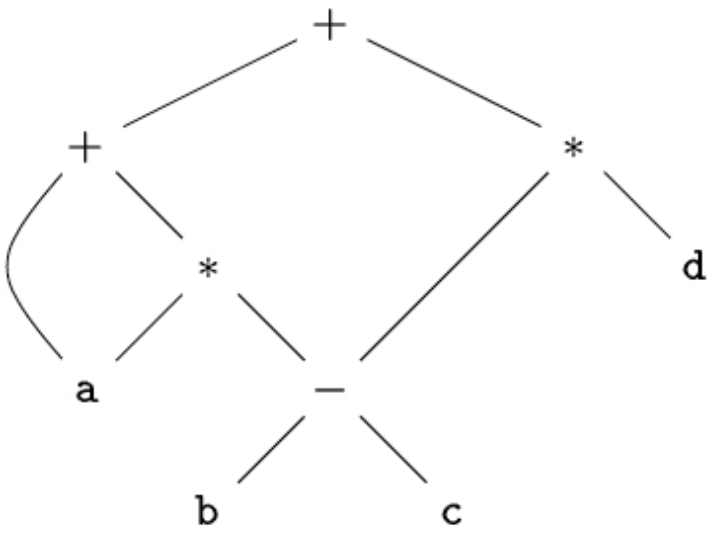
- Nodes with “similar” treatment for translation and type checking can be grouped

Concrete Syntax	Abstract Syntax
=	**assign**
||	**cond**
&&	**cond**
== !=	**rel**
< <= >= >	**rel**
+ -	**op**
* / %	**op**
!	**not**
$-_{unary}$	**minus**
[ ]	**access**

Direct Acyclic Graph (DAG)

A Direct Acyclic Graph (DAG) can be considered a compacted form of an Abstract Syntax Tree where common terms are not repeated. The result is that "leaves" will have more than one parent resulting in a graph rather than a tree structure

Consider the case of the expression $a + a * (b - c) + (b - c) * d$



DAG generation

How to generate it

The derivation of a DAG is much similar to that of a AST. In particular it is enough to revise the implementation of the `Node` method to avoid the replications of nodes

Let's recall the SDD for simple expressions...

Three Address Code

The term "three-address code" comes from instructions of the general form $x = y\ op\ z$ with three addresses (two for the operands and one for the result

In "three-address code" operations there is at most one operator on the right side of each single instruction.
Consider the expression: `x+y*z` the codification will look like ...

Building blocks

Three address code is built from two concepts: addresses, instructions.

Three Address Code

The term “three-address code” comes from instructions of the general form $x = y\ op\ z$ with three addresses (two for the operands and one for the result

In “three-address code” operations there is at most one operator on the right side of each single instruction.
Consider the expression: `x+y*z` the codification will look like . . .

Building blocks

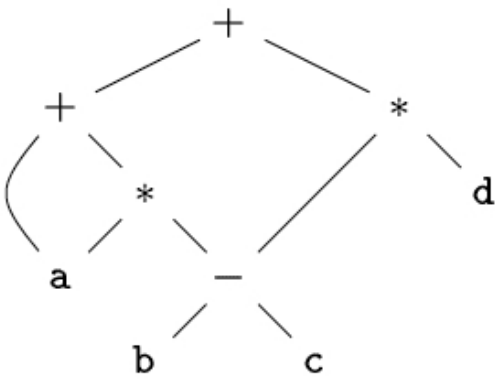
Three address code is built from two concepts: addresses, instructions.

Founding concepts

Addresses

- name
- constant
- compiler generated temporary

Three address codes are linearized representation of a syntax tree or DAG



(a) DAG

```
t1 = b - c
t2 = a * t1
t3 = a + t2
t4 = t1 * d
t5 = t3 + t4
```

(b) Three-address code

Founding concepts

Instructions

- assignment (with binary and unary operators) – e.g. $x = y \; op \; z$, $x = op \; y$
- copy instructions – e.g. $x = y$
- unconditional jump – e.g. `goto` L
- conditional jump with boolean – `if` x `goto` L, `ifFalse` x `goto` L
- conditional jump with relational operators – `if` x *relop* y `goto` L
- procedure calls and returns – e.g. `param x`, `call p,n`, and `y = call p,n`
- indexed copy instructions – e.g. `x=y[i]` and `x[i]=y`)
- Address and pointer assignment – e.g. `x=&y`, `x=*y`, `*x=y`)

Three address code representation and storage

Let's provide a translation for the following code fragment:

```
do
  i=i+1;
while (a[i] < v);
```

Data structures to represent the intermediate code (see book):

- Quadruples – includes results
- Triples
- Indirect Triples
- Static Single-Assignment (SSA) form

Three address code representation and storage

Quadruples example:

```
t1 = minus c
t2 = b * t1
t3 = minus c
t4 = b * t3
t5 = t2 + t4
 a = t5
```

(a) Three-address code

	op	arg_1	arg_2	*result*
0	minus	c		t_1
1	*	b	t_1	t_2
2	minus	c		t_3
3	*	b	t_3	t_4
4	+	t_2	t_4	t_5
5	=	t_5		a
	...			

(b) Quadruples

ToC

1. Semantic Analysis: the problem
2. Syntax Directed Definitions
3. Syntax Directed Translation Schemes
4. Preliminaries
5. Types
6. Control Flow

Types and Declarations

Types establish sets in which program elements can get their values. Two main activities related to compiling:

- Type Checking uses logical rules to reason about the behaviour of program at run time
- Translation Applications in which type related information are useful to determine the memory space needed for names at run-time, to compute address denoted by array reference, to apply conversions, to determine the operators to apply . . .

Type Expression

Type Expressions

A type expression is either a basic type of is formed by applying an operator, called type constructor, to a type expression. E.g. `int[2][3]`

inductive constructions of types expressions

- A basic type is a type expression (generally languages include basic types such as – boolean, char, integer, float, void, double, . . .)
- A type name is a type expression
- The array operator can be applied to a type expression to form a new type expression
- A record form a type expression from a list of type expressions
- The function operator ($\rightarrow$) can be used to define a function from a type s to type t
- The Cartesian product for two type expressions results in a new type expression

Declarations

Let's consider a simplified grammar for declarations:

$$D \rightarrow T\ \textbf{id}; D \ \mid\ \epsilon \qquad T \rightarrow BC \ \mid\ \textbf{record}\ '\{'\ D\ '\}'$$

$$B \rightarrow \textbf{int} \ \mid\ \textbf{float} \qquad C \rightarrow \epsilon \ \mid\ [\textbf{num}]C$$

Types and storage allocation

Worth to be mentioned:

- Relative addresses can be assigned at compile time
- Addressing constraints of the target machine influence assignment of addresses

Types and storage allocation for sequence of declarations

P	$\rightarrow$		{	offset=0}
		D		
D	$\rightarrow$	T **id**;	{	top.put(**id**.lexeme,T.type,offset);
				offset = offset + T.width; }
		D_1		
	|	ϵ		
T	$\rightarrow$	B	{	t=B.type; w=B.width; }
		C	{	T.type = C.type; T.width = C.width; }
	|	**record** $'\{'$	{	Env.push(top); top = new Env();
				Stack.push(offset); offset=0; }
		$D\ '\}'$	{	T.type=record(top); T.width=offset;
				top=Env.pop(); offset=Stack.pop(); }
B	$\rightarrow$	**int**	{	B.type=integer; B.width=4; }
	|	**float**	{	B.type=float; B.width=8; }
C	$\rightarrow$	[**num**]C_1	{	C.type = array(num.value,C_1.type);
				C.width=num.value $\times$ C_1.width; }
	|	ϵ	{	C.type = t; C.width = w; }

Translation of Expressions

In the translation of an expression we need to represent the code for the expression and the address in which the computed value will be stored. Therefore let's consider an excerpt for the usual expression grammar:

$$S \rightarrow \mathbf{id} = E \quad E \rightarrow E_1 + E_2 | - E_1 | (E_1) | \mathbf{id}$$

SDD for three address code translation

S	$\rightarrow$	**id** $= E$	S.code=E.code || gen(top.get(**id**.lexeme) '=' E.addr)
E	$\rightarrow$	$E_1 + E_2$	E.addr=**new** Temp() E.code = E_1.code || E_2.code || gen(E.addr '=' E_1.addr '+' E_2.addr)
	|	$-E_1$	E.addr=**new** Temp() E.code = E_1.code || gen(E.addr '=' '**minus**' E_1.addr)
	|	(E_1)	E.addr= E_1.addr, Ecode =E_1.code
	|	**id**	E.addr =top.get(**id**.lexeme), E.code= ' '

Consider the expression "a=b+-c" and derive the three address code translation applying the semantic rules defined

Type Checking

Type Checking ensures that the type of a construct matches that expected by its content. In:

if (*expr*) *stmt*

the expression *expr* is expected to have type **boolean**

Type Checking

- Type checking rules follow the operator/operand structure of the abstract syntax tree
- e.g. the operator **rel** represents the operator $\leq$ (and other relational ones such as $<$, $==$, $>$ and $\geq$)
- the type rule for **rel** says that the two operands must have the same type and the resulting type is **boolean**
- a synthesized attribute *type* may be used for implementing the rule in an SDD:

if ($E_1.type == E_2.type$) $E.type = \text{boolean}$;
else error;

Type Checking

To do type checking is necessary to assign a type expression to each component of the source program. Then a set of logical rules (type system) are defined to check if any non conformity is spotted.
Type checking can take two forms:

- type synthesis
- type inference

Type Synthesis

In type synthesis the type of an expression is derived from those of its sub-expressions. Names need to be declared before usage.
A typical rule will look like the following one:

if *f has type $s \rightarrow t$* ***and*** *x has type s,*
then *expression $f(x)$ has type t*

e.g. consider the case of $E_1 + E_2$

Type inference

With type inference the type of a construct is determined from the way it is used.

A typical rule for the type inference has the form:

if *$f(x)$ is an expression,*
then *for some α and β, f has type $\alpha \rightarrow \beta$* ***and*** *x has type α*

Type conversion

Consider the expression $a = b + c$ where the variable do not necessarily have the same type....managed via type conversion

All languages have their specific rules defined for conversion:

- narrowing
- widening

Conversion can be implicit or explicit. Implicit conversions, also called coercions, are generally limited to widening. Explicit conversions are consequence of statements included by the programmer (cast).

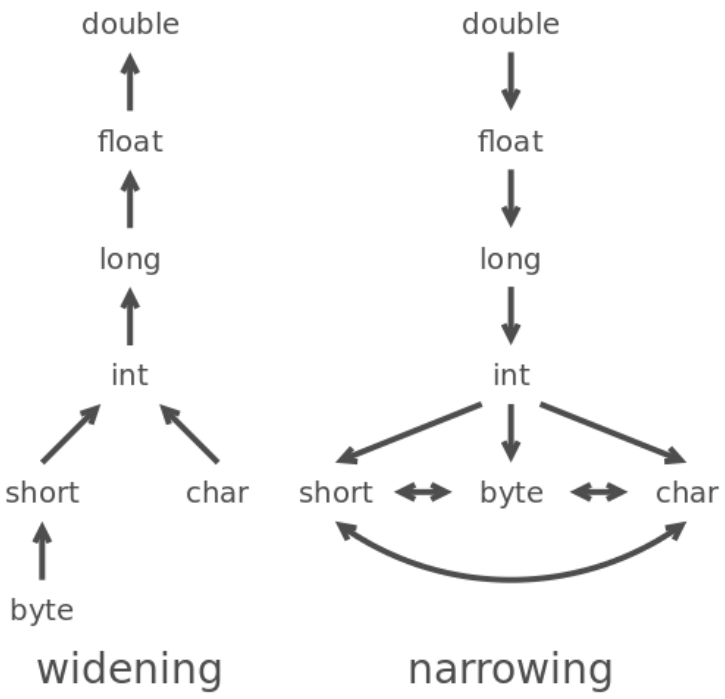
Type conversion

Consider the expression $a = b + c$ where the variable do not necessarily have the same type....managed via type conversion

All languages have their specific rules defined for conversion:

- narrowing
- widening

Conversion can be implicit or explicit. Implicit conversions, also called coercions, are generally limited to widening. Explicit conversions are consequence of statements included by the programmer (cast).



Type conversion

To define semantic actions for type checking two auxiliary functions are defined:

- $max(t_1, t_2)$: takes two types and return the maximum
- $widen(a, t, w)$ – where a is an address, while t and w are types: generate type conversion if needed to widen an address a of type t into a value of type w.

$E \rightarrow E_1 + E_2$ { E.type = max(E_1.type,E_2.type);
a_1 = widen (E_1.addr,E_1.type,E.type)
a_2 = widen (E_2.addr,E_2.type,E.type)
E.addr = **new** Temp();
gen(E.addr '=' a_1 '+' a_2); }

Type conversion

To define semantic actions for type checking two auxiliary functions are defined:

- $max(t_1, t_2)$: takes two types and return the maximum
- $widen(a, t, w)$ – where a is an address, while t and w are types: generate type conversion if needed to widen an address a of type t into a value of type w.

$$
\begin{array}{lcll}
E & \rightarrow & E_1 + E_2 & \{ \quad \text{E.type} = \max(\text{E}_1\text{.type},\text{E}_2\text{.type}); \\
 & & & \quad \text{a}_1 = \text{widen } (\text{E}_1\text{.addr},\text{E}_1\text{.type},\text{E.type}) \\
 & & & \quad \text{a}_2 = \text{widen } (\text{E}_2\text{.addr},\text{E}_2\text{.type},\text{E.type}) \\
 & & & \quad \text{E.addr} = \textbf{new } \text{Temp}(); \\
 & & & \quad \text{gen}(\text{E.addr '=' a}_1 \text{ '+' a}_2); \quad \}
\end{array}
$$

ToC

1. Semantic Analysis: the problem
2. Syntax Directed Definitions
3. Syntax Directed Translation Schemes
4. Preliminaries
5. Types
6. Control Flow

Control Flow

Boolean expression are the building block for influencing the flow of a program. They are manipulated to:

- Alter the flow of control – like in `if` $(E)S$
- Compute logical values

Two different approaches to evaluation:

- Eager
- Lazy

Short-Circuit Code

- Boolean operators ||, && and ! translate into jumps
- The operators do not appear on the code
- The value of a boolean expression is represented by a position in the code sequence

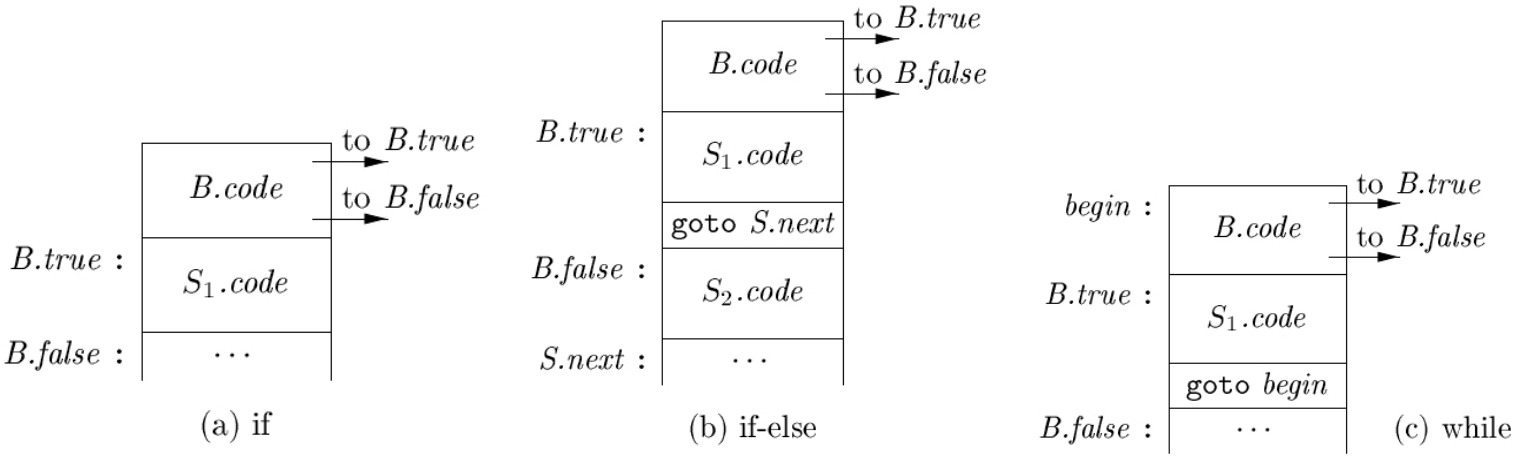
```
if (x < 100 || x > 200 && x != y) x = 0;
```

is translated to

```
        if x < 100 goto L2
        ifFalse x > 200 goto L1
        ifFalse x != y goto L1
L2:     x = 0
L1:     ...
```

Translation using jumping code

- Statements and Boolean Expressions have a synthesised attribute *code* that is a string containing the translated code
- Boolean Expressions have two inherited attributes *true* and *false* representing labels to which the control flows if the expression is true or false, respectively



Control Flow – commands

P	$\rightarrow$	S	S.next=newlabel(), P.code = S.code || label(S.next)
S	$\rightarrow$	**assign**	S.code=**assign**.code
S	$\rightarrow$	**if** (B) S_1	B.true=newlabel(), B.false=S_1.next=S.next S.code=B.code||label(B.true)||S_1.code
S	$\rightarrow$	**if** (B) S_1 **else** S_2	B.true=newlabel(), B.false=newlabel() S_1.next=S_2.next=S.next S.code=B.code||label(B.true)||S_1.code ||gen('`goto`' S.next)|| label(B.false)|| S_2.code
S	$\rightarrow$	**while** (B) S_1	begin=newlabel(), B.true=newlabel() B.false=S.next, S_1.next=begin S.code=label(begin)||B.code||label(B.true)||S_1.code ||gen('`goto`' begin)
S	$\rightarrow$	S_1 S_2	S_1.next=newlabel(), S_2.next=S.next S.code = S_1.code||label(S_1.next)||S_2.code

Control Flow – Boolean expressions

B	$\rightarrow$	$B_1 \| \| B_2$	B_1.true=B.true B_1.false=newlabel() B_2.true = B.true B_2.false=B.false B.code = B_1.code || label(B_1.false)|| B_2.code
B	$\rightarrow$	$B_1 \&\& B_2$	B_1.true=newlabel() B_1.false=B.false B_2.true = B.true B_2.false=B.false B.code = B_1.code || label(B_1.true)|| B_2.code
B	$\rightarrow$	E_1 **rel** E_2	B.code = E_1.code||E_2.code || gen('`if`' E_1.addr **rel**.op E_2.addr '`goto`' B.true) || gen('`goto`' B.false)
B	$\rightarrow$	**true**	B.code=gen('`goto`' B.true)
B	$\rightarrow$	**false**	B.code=gen('`goto`' B.false)

Control Flow – commands

Let's translate the following program:

```
if (x != y && x == z) x = y + z;
```